Flutter Backend Integration Guide

What is included

- `flutter_integration_kit/`: a reusable Flutter starter that matches this backend structure.
- `docs/flutter-integration/flutter-backend-integration-en.pdf`: English PDF export.
- `docs/flutter-integration/flutter-backend-integration-ar.pdf`: Arabic PDF export.

Backend contract used by the starter

- API base path: `/api/v1`
- Local development port in this repo: `3000`
- OpenAPI JSON: `/api/v1/docs/openapi.json`
- Login returns `accessToken` in the JSON body.
- Refresh and logout depend on the `refreshToken` cookie.
- Avatar upload field name: `avatar`
- Product images upload field name: `images`
- Product image and avatar URLs are returned as relative paths such as `/uploads/products/...`

Step 1: run the backend first

Use the backend from this repository as the source of truth while wiring Flutter.

```
npm run dev
```

Confirm these URLs:

- Browser docs UI: `http://localhost:3000/api/v1/docs`
- Raw OpenAPI JSON: `http://localhost:3000/api/v1/docs/openapi.json`

Step 2: open your Flutter mobile project

This repository does not contain the Flutter project itself, so integrate the starter into your separate Flutter app.

If you already have a Flutter app:

1. Copy the contents of `flutter_integration_kit/lib/` into your Flutter app `lib/` folder, or keep it as a feature module and rename imports as needed.

2. Merge `pubspec.yaml` dependencies from `flutter_integration_kit/pubspec.yaml`.
3. Run `flutter pub get`.

If you are starting from scratch:

```
flutter create ai_rent_mobile
cd ai_rent_mobile
flutter pub add dio dio_cookie_manager cookie_jar path path_provider flutter_secure_storage
```

Then copy the starter files into the new project.

Step 3: configure the correct API base URL

Use `--dart-define` so each environment can point to a different backend.

Recommended values:

- Android emulator: `http://10.0.2.2:3000/api/v1`
- iOS simulator: `http://127.0.0.1:3000/api/v1`
- Physical device on the same Wi-Fi: `http://YOUR_LOCAL_IP:3000/api/v1`

Example:

```
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:3000/api/v1
```

The starter reads this value from `AppConfig.fromEnvironment()`.

Step 4: allow HTTP traffic in development

If you are testing against local HTTP instead of HTTPS, mobile platforms need extra configuration.

Android:

In `android/app/src/main/AndroidManifest.xml`, set:

```
<application
    android:label="ai_rent_mobile"
    android:usesCleartextTraffic="true">
</application>
```

iOS:

In `ios/Runner/Info.plist`, add an ATS exception for local development:

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSAllowsArbitraryLoads</key>
  <true/>
```

```
</dict>
```

Use this only for local development. Move to HTTPS in staging and production.

Step 5: bootstrap dependencies in `main.dart`

The starter already includes a clean bootstrap flow:

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final dependencies = await AppDependencies.bootstrap();
  runApp(IntegrationStarterApp(dependencies: dependencies));
}
```

`AppDependencies` builds:

- `AppConfig`
- `SessionStore`
- persistent `CookieJar`
- `ApiClient`
- one repository per backend module

Step 6: wire authentication correctly

This backend uses a split token strategy:

1. `POST /auth/login` returns the access token in the response body.
2. The same response also sets the `refreshToken` cookie.
3. Protected requests use the `Authorization: Bearer <token>` header.
4. When the access token expires, the Flutter client calls `/auth/refresh-token` using the stored cookie.

The starter already handles this with:

- `SessionStore` for the access token
- `PersistCookieJar` for the refresh cookie
- `ApiClient` interceptor that refreshes the access token automatically for retryable protected requests

Typical login usage:

```
await dependencies.auth.login(
  email: emailController.text.trim(),
  password: passwordController.text,
);
```

```
final profile = await dependencies.users.getMyProfile();
```

Step 7: connect public screens first

Start with endpoints that do not need authentication. This gives you a fast smoke test.

Examples:

```
final categories = await dependencies.categories.listCategories();

final products = await dependencies.products.listProducts(
  page: 1,
  limit: 10,
  search: 'camera',
);
```

Suggested order:

1. Categories list
2. Public products list
3. Product details
4. Public owner profile

Step 8: connect protected renter and owner flows

After login works, wire the protected modules in this order:

1. Profile
2. Wishlist
3. Rental availability
4. Rental creation
5. My bookings and my requests
6. Notifications
7. Reviews

Example rental availability:

```
final result = await dependencies.rentals.checkAvailability(
  productId: productId,
  startDate: startDate,
  endDate: endDate,
  rentalPeriodType: 'daily',
  quantity: 1,
);
```

Example create rental:

```
await dependencies.rentals.createRental(
  productId: productId,
  startDate: startDate,
  endDate: endDate,
  rentalPeriodType: 'daily',
  quantity: 1,
  renterNotes: 'Please confirm pickup time.',
);
```

Step 9: connect file uploads the right way

Two endpoints use multipart form data:

- `POST /users/upload-avatar`
- `POST /products/{id}/images`

The exact field names are already correct in the starter:

- avatar: `avatar`
- product images: `images`

Examples:

```
await dependencies.users.uploadAvatar(filePath);

await dependencies.products.uploadProductImages(
  productId: productId,
  filePaths: selectedImagePaths,
);
```

Note:

- avatar size limit: 2 MB
- product image size limit: 5 MB each
- product image count limit: 10 files

Step 10: normalize returned image URLs

The backend returns relative paths such as:

```
/uploads/avatars/example.jpg
/uploads/products/example.jpg
```

In Flutter, turn them into absolute URLs before showing them:

```
final imageUrl = dependencies.config.resolveServerPath(rawPathFromApi);
```

Step 11: connect notifications and unread state

Notifications endpoints:

- `GET /notifications`
- `GET /notifications/unread-count`
- `PUT /notifications/{id}/read`
- `PUT /notifications/read-all`

Typical flow:

1. Load unread count on app start or after login.
2. Open the notifications screen.
3. Mark a single notification as read when the user opens it.
4. Refresh the unread badge.

Example:

```
await dependencies.notifications.markAsRead(notificationId);
final unread = await dependencies.notifications.getUnreadCount();
```

Step 12: connect admin screens only for admin users

Admin endpoints live under `/admin` and require both:

- valid access token
- user role = `admin`

Suggested admin integration order:

1. Dashboard
2. Users
3. Products moderation
4. Rentals
5. Reports

Example:

```
final dashboard = await dependencies.admin.getDashboard();
final pendingProducts = await dependencies.admin.getProducts(
  isApproved: false,
  page: 1,
  limit: 20,
);
```

Step 13: recommended folder structure inside your Flutter app

Keep the separation of concerns from the starter:

```
lib/
  src/
    config/
    core/
      http/
      storage/
      models/
    features/
      auth/
      users/
      categories/
      products/
      rentals/
      reviews/
      wishlist/
      recommendations/
      behavior/
      notifications/
      admin/
```

This keeps the project easier to debug, extend, and test.

Step 14: verification checklist

Run these checks in order:

1. Flutter app can load categories from the backend.
2. Login stores the access token and refresh cookie.
3. `/users/me` works after login.
4. Public product list and product details load correctly.
5. Relative image paths render as full URLs.
6. Wishlist add and remove both work.
7. Rental availability and rental creation both work.
8. Notifications unread count decreases after marking a notification as read.
9. Admin endpoints work only for admin accounts.
10. Logout clears the local access token and cookie state.

Backend endpoint groups covered by the starter

- Auth: register, login, refresh, logout, forgot password, reset password
- Users: profile, update profile, change password, upload avatar, public user data

• Categories: list, detail, create, update, delete

• Products: list, detail, create, update, update status, upload images, delete image, delete product

• Rentals: create, list bookings, list owner requests, detail, approve, reject, cancel, start, complete, availability

• Reviews: create, list product reviews, update, reply, delete

• Wishlist: list, add, remove

• Recommendations: personalized and similar products

• Behavior: track event

• Notifications: list, unread count, mark one read, mark all read

• Admin: dashboard, users, products, rentals, reports

Final note

Because the Flutter project itself is not present in this repository, this deliverable is a backend-aligned integration starter plus a step-by-step guide. Copy the starter into your mobile codebase, then connect each screen to the matching repository method shown above.